

National Institute of Technology, Warangal



Talk With Your Hands

Group No: 24

Prisha Behera 23CSB0F26

Dharani Murugan 23CSB0F27

Advaitha Patti 23CSB0F29

Department of Computer Science and Engineering

Talk With Your Hands

Abstract:

This project aims to build a system that can recognize American Sign Language (ASL) finger spelling and translate it into written English using machine learning. The main goal is to help people who are hearing- or speech-impaired communicate more easily with others by converting hand-signed letters into text. We will use Convolutional Neural Networks (CNN) to learn the visual patterns of different ASL letter signs and recognize them from images or videos. The system will process sequences of letters to form complete words and sentences. We will train our model using images of real ASL hand signs and use tools like OpenCV to detect hands from a camera in real-time. The final model will be able to recognize letters quickly and accurately, even in different lighting conditions or backgrounds. This system will demonstrate how artificial intelligence can be used to improve communication accessibility for people with special needs and can be expanded in the future to include dynamic signs, additional gestures, and other sign languages.

Introduction

1.1 The Communication Divide

Communication is the cornerstone of human interaction, enabling the exchange of ideas, emotions, and information that forms the bedrock of our societies. However, for millions of people around the world with hearing and speech impairments, this fundamental aspect of life presents significant challenges. The World Health Organization estimates that over 5% of the world's population – or 430 million people – require rehabilitation to address their ‘disabling’ hearing loss. This communication barrier can lead to social isolation, and limit access to education, employment, and essential services. The deaf and hard-of-hearing community primarily relies on sign language, a rich and expressive visual language. Yet, the vast majority of the hearing population does not understand it, creating a profound communication gap that hinders seamless interaction between the two groups.

1.2 Bridging the Gap with Technology

Traditionally, this communication gap has been bridged by human interpreters. While invaluable, interpreters are not always available, can be costly, and their presence may not be practical for spontaneous, everyday conversations. The advent of Artificial Intelligence, particularly in the fields of Computer Vision and Machine Learning, offers a powerful and accessible alternative to dissolve these barriers. By developing systems that can understand and translate sign language in real-time, we can empower individuals with hearing impairments to communicate more freely and effectively with the world around them, fostering a more inclusive and connected society.

1.3 Project Vision: A Real-Time Sign Language Converter

This project, "Sign-Language-To-Text-and-Speech-Conversion," aims to develop an intelligent system capable of recognizing alphabetic gestures from American Sign Language (ASL) and converting them into text and audible speech. The vision is to create a practical tool that can be used on a standard computer with a webcam, allowing for intuitive and real-time communication. The system will capture video input of a user's hand gestures, process the images to identify the sign being made, and display the corresponding text on the screen while simultaneously vocalizing it through text-to-speech synthesis.

1.4 Technical Approach and Methodology

To achieve this, the project leverages a sophisticated technical stack rooted in modern machine learning practices. The core of the recognition engine is a Convolutional Neural Network (CNN), a deep learning architecture exceptionally well-suited for image classification tasks. The methodology for this project can be broken down into several key stages:

1. **Data Collection and Preparation:** A comprehensive dataset of images for each letter of the ASL alphabet (A-Z) was gathered. This data is crucial for training a robust and accurate model. The images were carefully organized and pre-processed to ensure consistency and quality.
2. **Model Training:** The CNN model was trained on this extensive dataset. During training, the model learned to identify the intricate features and patterns that distinguish one sign from another. The model architecture, ``cnn8grps_rad1_model.h5``, was designed to be both accurate and computationally efficient.
3. **Real-Time Prediction:** The trained model is deployed in a Python application that accesses a live video feed from a webcam. For each frame, the application performs image processing to isolate the hand gesture, feeds it to the CNN model for prediction, and receives the recognized character as output.
4. **Text and Speech Conversion:** The recognized characters are aggregated to form words and sentences, which are displayed on a user interface. Concurrently, a text-to-speech (TTS) engine is used to convert the textual output into clear, audible speech, completing the communication cycle.

1.5 Project Scope and Objectives

The primary objective of this project is to build a proof-of-concept system that demonstrates the feasibility of real-time ASL alphabet recognition. The scope is specifically focused on static alphabetic gestures (A-Z), laying the foundational work for future expansion to include dynamic signs, words, and phrases. The key goals are:

- To design and implement a robust data collection pipeline for sign language gestures.
- To build and train a highly accurate CNN model for gesture classification.
- To develop a user-friendly application that integrates video capture, model prediction, and text-to-speech output in real-time.
- To create a system that is accessible and can run on standard consumer hardware.

1.6 Document Structure

This document provides a comprehensive overview of the entire project lifecycle. Following this introduction, the report is structured as follows: Section 2, the Vision Scope Document, outlines the project's high-level goals and stakeholder needs. Section 3 presents the Project Libre Document for project management details. Sections 4 and 5 detail the Usecase and SRS Documents, respectively, defining the system's functional and non-functional requirements. The subsequent sections delve into the system's design, architecture, source code, and testing methodologies, providing a complete and transparent account of the development process.

Source Code

Module 1 – Data Collection

Files:

- data_collection_binary.py
- data_collection_final.py

Purpose:

- Captures real-time webcam frames.
- Detects hand landmarks using cvzone + mediapipe.
- Generates skeleton images on a white background.
- Saves them in structured folders for each sign/letter.

Output:

- Dataset of images (used to train CNN model).

Module 2 – Model Prediction (without GUI)

File:

- prediction_wo_gui.py

Purpose:

- Loads the pre-trained CNN model (.h5).
- Processes live webcam input.
- Generates skeleton images and predicts the corresponding letter.
- Displays prediction in the console for quick testing/debugging.

Output:

- Console-based sign-to-letter predictions.

Module 3 – GUI Application

File:

- final_pred.py

Purpose:

- Provides a full-fledged GUI using **Tkinter**.

- Displays live camera feed and predictions.
- Builds full sentences.
- Offers **word suggestions** and **Text-to-Speech (TTS)** functionality.

Output:

- Interactive real-time application that converts signs into text and speech.

data_collection_binary.py

```
import cv2

from cvzone.HandTrackingModule import HandDetector
from cvzone.ClassificationModule import Classifier

import numpy as np

import os, os.path

from keras.models import load_model

import traceback

#model = load_model('C:\\Users\\PycharmProjects\\pythonProject\\cnn9.h5')

capture = cv2.VideoCapture(0)

hd = HandDetector(maxHands=1)

hd2 = HandDetector(maxHands=1)

# #training data

# count = len(os.listdir("D://sign2text_dataset_2.0/Binary_imgs//A"))

#testing data

count = len(os.listdir("D://test_data_2.0//Gray_imgs//A"))

p_dir = "A"

c_dir = "a"

offset = 30

step = 1

flag=False
```

```

suv=0

#C:\Users\PycharmProjects\pythonProject
white=np.ones((400,400),np.uint8)*255

cv2.imwrite("C:\\Users\\devansh raval\\PycharmProjects\\pythonProject\\white.jpg",white)

while True:

    try:

        __, frame = capture.read()

        frame = cv2.flip(frame, 1)

        hands= hd.findHands(frame, draw=False, flipType=True)

        img_final=img_final1=img_final2=0

        if hands:

            hand = hands[0]

            x, y, w, h = hand['bbox']

            image = frame[y - offset:y + h + offset, x - offset:x + w + offset]

            #image1 = imgg[y - offset:y + h + offset, x - offset:x + w + offset]

            roi = image    #rgb image without drawing

            # roi1 = image1  #rdb image with drawing

            # #for simple gray image without draw

            gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)

            blur = cv2.GaussianBlur(gray, (1, 1), 2)

            #

            # #for binary image

            gray2 = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)

            blur2 = cv2.GaussianBlur(gray2, (5, 5), 2)

            th3 = cv2.adaptiveThreshold(blur2, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY_INV, 11, 2)

```

```

ret, test_image = cv2.threshold(th3, 27, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)

#

#

test_image1=blur

img_final1 = np.ones((400, 400), np.uint8) * 148

h = test_image1.shape[0]

w = test_image1.shape[1]

img_final1[((400 - h) // 2):((400 - h) // 2) + h, ((400 - w) // 2):((400 - w) // 2) + w] =
test_image1

img_final = np.ones((400, 400), np.uint8) * 255

h = test_image.shape[0]

w = test_image.shape[1]

img_final[((400 - h) // 2):((400 - h) // 2) + h, ((400 - w) // 2):((400 - w) // 2) + w] =
test_image

hands = hd.findHands(frame, draw=False, flipType=True)

if hands:

# #print("-----lmList=",hands[1])

hand = hands[0]

x, y, w, h = hand['bbox']

image = frame[y - offset:y + h + offset, x - offset:x + w + offset]

white = cv2.imread("C:\\Users\\PycharmProjects\\pythonProject\\white.jpg")

# img_final=img_final1=img_final2=0

handz = hd2.findHands(image, draw=False, flipType=True)

if handz:

    hand = handz[0]

    pts = hand['lmList']

```



```

# x1,y1,w1,h1=hand['bbox']

os = ((400 - w) // 2) - 15
os1 = ((400 - h) // 2) - 15
for t in range(0, 4, 1):
    cv2.line(white, (pts[t][0] + os, pts[t][1] + os1), (pts[t + 1][0] + os, pts[t + 1][1]
+ os1),
              (0, 255, 0), 3)
for t in range(5, 8, 1):
    cv2.line(white, (pts[t][0] + os, pts[t][1] + os1), (pts[t + 1][0] + os, pts[t + 1][1]
+ os1),
              (0, 255, 0), 3)
for t in range(9, 12, 1):
    cv2.line(white, (pts[t][0] + os, pts[t][1] + os1), (pts[t + 1][0] + os, pts[t + 1][1]
+ os1),
              (0, 255, 0), 3)
for t in range(13, 16, 1):
    cv2.line(white, (pts[t][0] + os, pts[t][1] + os1), (pts[t + 1][0] + os, pts[t + 1][1]
+ os1),
              (0, 255, 0), 3)
for t in range(17, 20, 1):
    cv2.line(white, (pts[t][0] + os, pts[t][1] + os1), (pts[t + 1][0] + os, pts[t + 1][1]
+ os1),
              (0, 255, 0), 3)
    cv2.line(white, (pts[5][0] + os, pts[5][1] + os1), (pts[9][0] + os, pts[9][1] +
os1), (0, 255, 0),
              3)
    cv2.line(white, (pts[9][0] + os, pts[9][1] + os1), (pts[13][0] + os, pts[13][1] +
os1), (0, 255, 0),
              3)

```

```

        cv2.line(white, (pts[13][0] + os, pts[13][1] + os1), (pts[17][0] + os, pts[17][1]
+ os1),
                (0, 255, 0), 3)

        cv2.line(white, (pts[0][0] + os, pts[0][1] + os1), (pts[5][0] + os, pts[5][1] +
os1), (0, 255, 0),
                3)

        cv2.line(white, (pts[0][0] + os, pts[0][1] + os1), (pts[17][0] + os, pts[17][1] +
os1), (0, 255, 0),
                3)

    for i in range(21):

        cv2.circle(white, (pts[i][0] + os, pts[i][1] + os1), 2, (0, 0, 255), 1)

    cv2.imshow("skeleton", white)

    # cv2.imshow("5", skeleton5)

    hands = hd.findHands(white, draw=False, flipType=True)

    if hands:

        hand = hands[0]

        x, y, w, h = hand['bbox']

        cv2.rectangle(white, (x - offset, y - offset), (x + w, y + h), (3, 255, 25), 3)

    image1 = frame[y - offset:y + h + offset, x - offset:x + w + offset]

    roi1 = image1 #rdb image with drawing

    #for gray image with drawings

    gray1 = cv2.cvtColor(roi1, cv2.COLOR_BGR2GRAY)

    blur1 = cv2.GaussianBlur(gray1, (1, 1), 2)

```

```

test_image2= blur1

img_final2= np.ones((400, 400), np.uint8) * 148

h = test_image2.shape[0]

w = test_image2.shape[1]

img_final2[((400 - h) // 2):((400 - h) // 2) + h, ((400 - w) // 2):((400 - w) // 2) + w] =
test_image2

#cv2.imshow("aaa",white)

# cv2.imshow("gray",img_final2)

cv2.imshow("binary", img_final)

# cv2.imshow("gray w/o draw", img_final1)

# img = img_final.reshape(1, 400, 400, 1)

# # print(model.predict(img))

# prob = np.array(model.predict(img)[0], dtype='float32')

# ch1 = np.argmax(prob, axis=0)

# prob[ch1] = 0

# ch2 = np.argmax(prob, axis=0)

# prob[ch2] = 0

# ch3 = np.argmax(prob, axis=0)

# prob[ch3] = 0

# ch1 = chr(ch1 + 65)

# ch2 = chr(ch2 + 65)

# ch3 = chr(ch3 + 65)

# frame = cv2.putText(frame, "Predicted " + ch1 + " " + ch2 + " " + ch3, (x - offset -
150, y - offset - 10),

#             cv2.FONT_HERSHEY_SIMPLEX,

#             1, (255, 0, 0), 1, cv2.LINE_AA)

#cv2.rectangle(frame, (x - offset, y - offset), (x + w, y + h), (3, 255, 25), 3)

```

```

# frame = cv2.putText(frame, "dir=" + c_dir + " count=" + str(count), (50,50),
#
#           cv2.FONT_HERSHEY_SIMPLEX,
#
#           1, (255, 0, 0), 1, cv2.LINE_AA)
cv2.imshow("frame", frame)

interrupt = cv2.waitKey(1)

if interrupt & 0xFF == 27:

    # esc key

    break

if interrupt & 0xFF == ord('n'):

    p_dir = chr(ord(p_dir) + 1)

    c_dir = chr(ord(c_dir) + 1)

    if ord(p_dir)==ord('Z')+1:

        p_dir="A"

        c_dir="a"

    flag = False

# #training data

# count = len(os.listdir("D://sign2text_dataset_2.0/Binary_imgs/" + p_dir + "/" ))

# test data

count = len(os.listdir("D://test_data_2.0/Gray_imgs/" + p_dir + "/" ))

if interrupt & 0xFF == ord('a'):

    if flag:

        flag=False

    else:

        suv=0

        flag=True

```

```

print("=====",flag)

if flag==True:

    if suv==50:

        flag=False

    if step%2==0:

        # #this is for training data collection

        # cv2.imwrite("D:\\sign2text_dataset_2.0\\Binary_imgs\\" + p_dir + "\\" +
c_dir + str(count) + ".jpg", img_final)

        # cv2.imwrite("D:\\sign2text_dataset_2.0\\Gray_imgs\\" + p_dir + "\\" + c_dir
+ str(count) + ".jpg", img_final1)

        #   cv2.imwrite("D:\\sign2text_dataset_2.0\\Gray_imgs_with_drawing\\"      +
p_dir + "\\" + c_dir + str(count) + ".jpg", img_final2)


        # this is for testing data collection

        # cv2.imwrite("D:\\test_data_2.0\\Binary_imgs\\" + p_dir + "\\" + c_dir +
str(count) + ".jpg",

        #       img_final)

        cv2.imwrite("D:\\test_data_2.0\\Gray_imgs\\" + p_dir + "\\" + c_dir + str(count) +
".jpg",

        img_final1)

        cv2.imwrite(

        "D:\\test_data_2.0\\Gray_imgs_with_drawing\\" + p_dir + "\\" + c_dir +
str(count) + ".jpg",

        img_final2)

        count += 1

        suv += 1

    step+=1

except Exception:

print("==",traceback.format_exc() )

```

```
capture.release()
cv2.destroyAllWindows()
```

data_collection_final.py

```
import cv2

from cvzone.HandTrackingModule import HandDetector

import numpy as np

import os as oss

import traceback

capture = cv2.VideoCapture(0)

hd = HandDetector(maxHands=1)

hd2 = HandDetector(maxHands=1)


count = len(oss.listdir("D:\\sign2text_dataset_3.0\\AtoZ_3.0\\A\\"))

c_dir = 'A'


offset = 15

step = 1

flag=False

suv=0


white=np.ones((400,400),np.uint8)*255

cv2.imwrite("C:\\Users\\PycharmProjects\\pythonProject\\white.jpg",white)

while True:

    try:

        _, frame = capture.read()

        frame = cv2.flip(frame, 1)
```

```

hands= hd.findHands(frame, draw=False, flipType=True)

white = cv2.imread("C:\\Users\\PycharmProjects\\pythonProject\\white.jpg")

if hands:

    hand = hands[0]

    x, y, w, h = hand['bbox']

    image =np.array( frame[y - offset:y + h + offset, x - offset:x + w + offset])

    handz,imz = hd2.findHands(image, draw=True, flipType=True)

    if handz:

        hand = handz[0]

        pts = hand['lmList']

        # x1,y1,w1,h1=hand['bbox']

        os=((400-w)//2)-15

        os1=((400-h)//2)-15

        for t in range(0,4,1):

cv2.line(white,(pts[t][0]+os,pts[t][1]+os1),(pts[t+1][0]+os,pts[t+1][1]+os1),(0,255,0),3)

        for t in range(5,8,1):

cv2.line(white,(pts[t][0]+os,pts[t][1]+os1),(pts[t+1][0]+os,pts[t+1][1]+os1),(0,255,0),3)

        for t in range(9,12,1):

cv2.line(white,(pts[t][0]+os,pts[t][1]+os1),(pts[t+1][0]+os,pts[t+1][1]+os1),(0,255,0),3)

        for t in range(13,16,1):

cv2.line(white,(pts[t][0]+os,pts[t][1]+os1),(pts[t+1][0]+os,pts[t+1][1]+os1),(0,255,0),3)

        for t in range(17,20,1):

cv2.line(white,(pts[t][0]+os,pts[t][1]+os1),(pts[t+1][0]+os,pts[t+1][1]+os1),(0,255,0),3)

```

```

cv2.line(white, (pts[5][0]+os, pts[5][1]+os1), (pts[9][0]+os, pts[9][1]+os1), (0,
255, 0), 3)

cv2.line(white, (pts[9][0]+os, pts[9][1]+os1), (pts[13][0]+os, pts[13][1]+os1),
(0, 255, 0), 3)

cv2.line(white, (pts[13][0]+os, pts[13][1]+os1), (pts[17][0]+os,
pts[17][1]+os1), (0, 255, 0), 3)

cv2.line(white, (pts[0][0]+os, pts[0][1]+os1), (pts[5][0]+os, pts[5][1]+os1), (0,
255, 0), 3)

cv2.line(white, (pts[0][0]+os, pts[0][1]+os1), (pts[17][0]+os, pts[17][1]+os1),
(0, 255, 0), 3)


skeleton0=np.array(white)

zz=np.array(white)

for i in range(21):
cv2.circle(white,(pts[i][0]+os,pts[i][1]+os1),2,(0 , 0 , 255),1)


skeleton1=np.array(white)


cv2.imshow("1",skeleton1)


frame = cv2.putText(frame, "dir=" + str(c_dir) + " count=" + str(count), (50,50),
cv2.FONT_HERSHEY_SIMPLEX,
1, (255, 0, 0), 1, cv2.LINE_AA)

cv2.imshow("frame", frame)

interrupt = cv2.waitKey(1)

if interrupt & 0xFF == 27:

# esc key

break

```



```

if interrupt & 0xFF == ord('n'):
    c_dir = chr(ord(c_dir)+1)
    if ord(c_dir)==ord('Z')+1:
        c_dir='A'

    flag = False

    count = len(oss.listdir("D:\\sign2text_dataset_3.0\\AtoZ_3.0\\" + (c_dir) + "\\"))

    if interrupt & 0xFF == ord('a'):
        if flag:
            flag=False
        else:
            suv=0
            flag=True

    print("=====",flag)

    if flag==True:

        if suv==180:
            flag=False

        if step%3==0:
            cv2.imwrite("D:\\sign2text_dataset_3.0\\AtoZ_3.1\\" + (c_dir) + "\\" + str(count) +
".jpg",
                        skeleton1)

            count += 1
            suv += 1

        step+=1

```

```

except Exception:

    print("==",traceback.format_exc() )

capture.release()

cv2.destroyAllWindows()

```

prediction_wo_gui.py

```

import math

import cv2

from cvzone.HandTrackingModule import HandDetector

import numpy as np

from keras.models import load_model

import traceback

model = load_model('/cnn8grps_rad1_model.h5')

white = np.ones((400, 400), np.uint8) * 255

cv2.imwrite("C:\\Users\\PycharmProjects\\pythonProject\\white.jpg", white)

capture = cv2.VideoCapture(0)

hd = HandDetector(maxHands=1)

hd2 = HandDetector(maxHands=1)

offset = 29

step = 1

flag = False

suv = 0

def distance(x, y):

    return math.sqrt(((x[0] - y[0]) ** 2) + ((x[1] - y[1]) ** 2))

def distance_3d(x, y):

    return math.sqrt(((x[0] - y[0]) ** 2) + ((x[1] - y[1]) ** 2) + ((x[2] - y[2]) ** 2))

```

```

bfh = 0

dicttt=dict()

count=0

kok=[]


while True:

    try:

        _, frame = capture.read()

        frame = cv2.flip(frame, 1)

        hands = hd.findHands(frame, draw=False, flipType=True)

        print(frame.shape)

        if hands:

            # #print("-----lmList=",hands[1])

            hand = hands[0]

            x, y, w, h = hand['bbox']

            image = frame[y - offset:y + h + offset, x - offset:x + w + offset]

            white = cv2.imread("white.jpg")

            # img_final=img_final1=img_final2=0

            handz = hd2.findHands(image, draw=False, flipType=True)

            if handz:

                hand = handz[0]

                pts = hand['lmList']

                # x1,y1,w1,h1=hand['bbox']


                os = ((400 - w) // 2) - 15

                os1 = ((400 - h) // 2) - 15

                for t in range(0, 4, 1):

```

```

cv2.line(white, (pts[t][0] + os, pts[t][1] + os1), (pts[t + 1][0] + os, pts[t + 1][1]
+ os1),
        (0, 255, 0), 3)
for t in range(5, 8, 1):
cv2.line(white, (pts[t][0] + os, pts[t][1] + os1), (pts[t + 1][0] + os, pts[t + 1][1]
+ os1),
        (0, 255, 0), 3)
for t in range(9, 12, 1):
cv2.line(white, (pts[t][0] + os, pts[t][1] + os1), (pts[t + 1][0] + os, pts[t + 1][1]
+ os1),
        (0, 255, 0), 3)
for t in range(13, 16, 1):
cv2.line(white, (pts[t][0] + os, pts[t][1] + os1), (pts[t + 1][0] + os, pts[t + 1][1]
+ os1),
        (0, 255, 0), 3)
for t in range(17, 20, 1):
cv2.line(white, (pts[t][0] + os, pts[t][1] + os1), (pts[t + 1][0] + os, pts[t + 1][1]
+ os1),
        (0, 255, 0), 3)

cv2.line(white, (pts[5][0] + os, pts[5][1] + os1), (pts[9][0] + os, pts[9][1] +
os1), (0, 255, 0),
        3)
cv2.line(white, (pts[9][0] + os, pts[9][1] + os1), (pts[13][0] + os, pts[13][1] +
os1), (0, 255, 0),
        3)
cv2.line(white, (pts[13][0] + os, pts[13][1] + os1), (pts[17][0] + os, pts[17][1]
+ os1),
        (0, 255, 0), 3)
cv2.line(white, (pts[0][0] + os, pts[0][1] + os1), (pts[5][0] + os, pts[5][1] +
os1), (0, 255, 0),
        3)

```

```
cv2.line(white, (pts[0][0] + os, pts[0][1] + os1), (pts[17][0] + os, pts[17][1] +
os1), (0, 255, 0),
```

```
3)
```

```
for i in range(21):
```

```
cv2.circle(white, (pts[i][0] + os, pts[i][1] + os1), 2, (0, 0, 255), 1)
```

```
cv2.imshow("2", white)
```

```
# cv2.imshow("5", skeleton5)
```

```
# #print(model.predict(img))
```

```
white = white.reshape(1, 400, 400, 3)
```

```
prob = np.array(model.predict(white)[0], dtype='float32')
```

```
ch1 = np.argmax(prob, axis=0)
```

```
prob[ch1] = 0
```

```
ch2 = np.argmax(prob, axis=0)
```

```
prob[ch2] = 0
```

```
ch3 = np.argmax(prob, axis=0)
```

```
prob[ch3] = 0
```

```
pl = [ch1, ch2]
```

```
#condition for [Aemnst]
```

```
l=[[5,2],[5,3],[3,5],[3,6],[3,0],[3,2],[6,4],[6,1],[6,2],[6,6],[6,7],[6,0],[6,5],[4,1],[1,0],[1,1],[6,3],
[1,6],[5,6],[5,1],[4,5],[1,4],[1,5],[2,0],[2,6],[4,6],[1,0],[5,7],[1,6],[6,1],[7,6],[2,5],[7,1],[5,4],
[7,0],[7,5],[7,2]]
```

```
if pl in l:
```

```

        if (pts[6][1] < pts[8][1] and pts[10][1] < pts[12][1] and pts[14][1] < pts[16][1]
and pts[18][1] <pts[20][1]):

    ch1=0

    #print("00000")


    #condition for [o][s]

    l=[[2,2],[2,1]]

    if pl in l:

        if (pts[5][0] < pts[4][0] ):

            ch1=0

            print("+++++")

            #print("00000")

            #condition for [c0][aemnst]

l=[[0,0],[0,6],[0,2],[0,5],[0,1],[0,7],[5,2],[7,6],[7,1]]

        pl=[ch1,ch2]

        if pl in l:

            if (pts[0][0]>pts[8][0] and pts[0][0]>pts[4][0] and pts[0][0]>pts[12][0] and
pts[0][0]>pts[16][0] and pts[0][0]>pts[20][0]) and pts[5][0] > pts[4][0]:

                ch1=2

                #print("22222")


            # condition for [c0][aemnst]

            l = [[6,0],[6,6],[6,2]]

            pl = [ch1, ch2]

            if pl in l:

                if distance(pts[8],pts[16])<52:

                    ch1 = 2

                    #print("22222")

```

```

##print(pts[2][1]+15>pts[16][1])

    # condition for [gh][bdfikruvw]

    l = [[1,4],[1,5],[1,6],[1,3],[1,0]]

    pl = [ch1, ch2]


    if pl in l:

        if pts[6][1] > pts[8][1] and pts[14][1] < pts[16][1] and pts[18][1]<pts[20][1]
and pts[0][0]<pts[8][0] and pts[0][0]<pts[12][0] and pts[0][0]<pts[16][0] and
pts[0][0]<pts[20][0]:

            ch1 = 3

            print("33333c")

            #con for [gh][l]

l=[[4,6],[4,1],[4,5],[4,3],[4,7]]

    pl=[ch1,ch2]

    if pl in l:

        if pts[4][0]>pts[0][0]:

            ch1=3

            print("33333b")

            # con for [gh][pqz]

l = [[5, 3],[5,0],[5,7], [5, 4], [5, 2],[5,1],[5,5]]

    pl = [ch1, ch2]

    if pl in l:

        if pts[2][1]+15<pts[16][1]:

            ch1 = 3

            print("33333a")

            # con for [l][x]

l = [[6, 4], [6, 1], [6, 2]]

    pl = [ch1, ch2]

```

```

if pl in l:
if distance(pts[4],pts[11])>55:
ch1 = 4
#print("44444")
# con for [l][d]
l = [[1, 4], [1, 6],[1,1]]
pl = [ch1, ch2]
if pl in l:
if (distance(pts[4], pts[11]) > 50) and (pts[6][1] > pts[8][1] and pts[10][1] <
pts[12][1] and pts[14][1] < pts[16][1] and pts[18][1] <pts[20][1]):
ch1 = 4
#print("44444")
# con for [l][gh]
l = [[3, 6], [3, 4]]
pl = [ch1, ch2]
if pl in l:
if (pts[4][0]<pts[0][0]):
ch1 = 4
#print("44444")

# con for [l][c0]
l = [[2, 2], [2, 5],[2,4]]
pl = [ch1, ch2]
if pl in l:
if (pts[1][0] < pts[12][0]):
ch1 = 4
#print("44444")

```



```

# con for [l][c0]

l = [[2, 2], [2, 5], [2, 4]]

pl = [ch1, ch2]

if pl in l:

if (pts[1][0] < pts[12][0]):

ch1 = 4

#print("44444")

# con for [gh][z]

l = [[3, 6],[3,5],[3,4]]

pl = [ch1, ch2]

if pl in l:

if (pts[6][1] > pts[8][1] and pts[10][1] < pts[12][1] and pts[14][1] < pts[16][1]
and pts[18][1] <pts[20][1]) and pts[4][1]>pts[10][1]:

ch1 = 5

print("55555b")

# con for [gh][pq]

l = [[3,2],[3,1],[3,6]]

pl = [ch1, ch2]

if pl in l:

if pts[4][1]+17>pts[8][1] and pts[4][1]+17>pts[12][1] and
pts[4][1]+17>pts[16][1] and pts[4][1]+17>pts[20][1]:

ch1 = 5

print("55555a")

# con for [l][pqz]

l = [[4,4],[4,5],[4,2],[7,5],[7,6],[7,0]]

pl = [ch1, ch2]

if pl in l:

if pts[4][0]>pts[0][0]:

```

```

ch1 = 5

#print("55555")

# con for [pqz][aemnst]

l = [[0, 2],[0,6],[0,1],[0,5],[0,0],[0,7],[0,4],[0,3],[2,7]]

pl = [ch1, ch2]

if pl in l:

    if pts[0][0]<pts[8][0] and pts[0][0]<pts[12][0] and pts[0][0]<pts[16][0] and
pts[0][0]<pts[20][0]:

        ch1 = 5

        #print("55555")

        # con for [pqz][yj]

        l = [[5, 7],[5,2],[5,6]]

        pl = [ch1, ch2]

        if pl in l:

            if pts[3][0]<pts[0][0]:

                ch1 = 7

                #print("77777")

                # con for [l][yj]

                l = [[4, 6],[4,2],[4,4],[4,1],[4,5],[4,7]]

                pl = [ch1, ch2]

                if pl in l:

                    if pts[6][1] < pts[8][1]:

                        ch1 = 7

                        #print("77777")

                        # con for [x][yj]

                        l = [[6, 7],[0,7],[0,1],[0,0],[6,4],[6,6] ,[6,5],[6,1]]

                        pl = [ch1, ch2]

```

```

if pl in l:
if pts[18][1] > pts[20][1]:
ch1 = 7
#print("77777")
# condition for [x][aemnst]
l = [[0,4],[0,2],[0,3],[0,1],[0,6]]
pl = [ch1, ch2]
if pl in l:
if pts[5][0]>pts[16][0]:
ch1 = 6
#print("66666")
# condition for [yj][x]
l = [[7, 2]]
pl = [ch1, ch2]
if pl in l:
if pts[18][1] < pts[20][1]:
ch1 = 6
#print("66666")
# condition for [c0][x]
l = [[2, 1],[2,2],[2,6],[2,7],[2,0]]
pl = [ch1, ch2]
if pl in l:
if distance(pts[8],pts[16])>50:
ch1 = 6
#print("66666")
# con for [l][x]
l = [[4, 6],[4,2],[4,1],[4,4]]

```

```

pl = [ch1, ch2]
if pl in l:
    if distance(pts[4], pts[11]) < 60:
        ch1 = 6
        #print("66666")
        #con for [x][d]
        l = [[1,4],[1,6],[1,0],[1,2]]
        pl = [ch1, ch2]
        if pl in l:
            if pts[5][0] - pts[4][0] - 15 > 0:
                ch1 = 6
                # con for [b][pqz]
                l
                =
[[5,0],[5,1],[5,4],[5,5],[5,6],[6,1],[7,6],[0,2],[7,1],[7,4],[6,6],[7,2],[5,0],[6,3],[6,4],[7,5],[7,2]]
                pl = [ch1, ch2]
                if pl in l:
                    if (pts[6][1] > pts[8][1] and pts[10][1] > pts[12][1] and pts[14][1] > pts[16][1]
and pts[18][1] > pts[20][1]):
                        ch1 = 1
                        print("111111")
                        # con for [f][pqz]
                        l = [[6, 1],[6,0],[0,3],[6,4],[2,2], [0,6],[6,2],[7, 6],[4,6],[4,1],[4,2], [0, 2], [7, 1],
[7, 4], [6, 6], [7, 2], [7, 5], [7, 2]]
                        pl = [ch1, ch2]
                        if pl in l:
                            if (pts[6][1] < pts[8][1] and pts[10][1] > pts[12][1] and pts[14][1] > pts[16][1]
and
                                pts[18][1] > pts[20][1]):
                                    ch1 = 1

```

```

print("111112")

l = [[6, 1], [6, 0],[4,2],[4,1],[4,6],[4,4]]

pl = [ch1, ch2]

if pl in l:

    if (pts[10][1] > pts[12][1] and pts[14][1] > pts[16][1] and
        pts[18][1] > pts[20][1]):

        ch1 = 1

        print("111112")

        # con for [d][pqz]

        fg=19

        #print("_____ch1=",ch1," ch2=",ch2)

        l = [[5,0],[3,4],[3,0],[3,1],[3,5],[5,5],[5,4],[5,1],[7,6]]

        pl = [ch1, ch2]

        if pl in l:

            if ((pts[6][1] > pts[8][1] and pts[10][1] < pts[12][1] and pts[14][1] < pts[16][1]
and
            pts[18][1] < pts[20][1]) and (pts[2][0]<pts[0][0]) and
pts[4][1]>pts[14][1]):

                ch1 = 1

                print("111113")

                l = [ [4, 1], [4, 2],[4, 4]]

                pl = [ch1, ch2]

                if pl in l:

                    if (distance(pts[4], pts[11]) < 50) and (pts[6][1] > pts[8][1] and pts[10][1] <
pts[12][1] and pts[14][1] < pts[16][1] and pts[18][1] < pts[20][1]):

                        ch1 = 1

                        print("1111993")

                        l = [[3, 4], [3, 0], [3, 1], [3, 5],[3,6]]

```

```

pl = [ch1, ch2]
if pl in l:
    if ((pts[6][1] > pts[8][1] and pts[10][1] < pts[12][1] and pts[14][1] < pts[16][1]
and
pts[18][1] < pts[20][1]) and (pts[2][0] < pts[0][0]) and pts[14][1]<pts[4][1]):
        ch1 = 1
        print("1111mmm3")
l = [[6, 6],[6, 4], [6, 1],[6,2]]
pl = [ch1, ch2]
if pl in l:
    if pts[5][0]-pts[4][0]-15<0:
        ch1 = 1
        print("1111140")
# con for [i][pqz]
l = [[5,4],[5,5],[5,1],[0,3],[0,7],[5,0],[0,2],[6,2],[7, 5], [7, 1], [7, 6], [7, 7]]
pl = [ch1, ch2]
if pl in l:
    if ((pts[6][1] < pts[8][1] and pts[10][1] < pts[12][1] and pts[14][1] < pts[16][1]
and
pts[18][1] > pts[20][1])):
        ch1 = 1
        print("111114")

# con for [yj][bfdi]
l = [[1,5],[1,7],[1,1],[1,6],[1,3],[1,0]]
pl = [ch1, ch2]
if pl in l:

```

```

        if (pts[4][0]<pts[5][0]+15) and ((pts[6][1] < pts[8][1] and pts[10][1] <
pts[12][1] and pts[14][1] < pts[16][1] and
        pts[18][1] > pts[20][1])):
            ch1 = 7
            print("111114lll;p")
            #con for [uvr]
            l = [[5,5],[5,0],[5,4],[5,1],[4,6],[4,1],[7,6],[3,0],[3,5]]
            pl = [ch1, ch2]
            if pl in l:
                if ((pts[6][1] > pts[8][1] and pts[10][1] > pts[12][1] and pts[14][1] < pts[16][1]
and
                pts[18][1] < pts[20][1])) and pts[4][1]>pts[14][1]:
                    ch1 = 1
                    print("111115")
                    # con for [w]
                    fg=13
                    l = [[3,5],[3,0],[3,6],[5,1],[4,1],[2,0],[5,0],[5,5]]
                    pl = [ch1, ch2]
                    if pl in l:
                        if not(pts[0][0]+fg < pts[8][0] and pts[0][0]+fg < pts[12][0] and pts[0][0]+fg <
pts[16][0] and pts[0][0]+fg < pts[20][0]) and not(pts[0][0] > pts[8][0] and pts[0][0] >
pts[12][0] and pts[0][0] > pts[16][0] and pts[0][0] > pts[20][0]) and distance(pts[4], pts[11])
< 50:
                            ch1 = 1
                            print("111116")
                            # con for [w]
                            l = [ [5, 0], [5, 5],[0,1]]
                            pl = [ch1, ch2]
                            if pl in l:

```

```

if pts[6][1]>pts[8][1] and pts[10][1]>pts[12][1] and pts[14][1]>pts[16][1]:
    ch1 = 1

    print("1117")

    #-----condn for 8 groups ends

    #-----condn for subgroups starts

    #

    if ch1 == 0:

        ch1='S'

        if pts[4][0] < pts[6][0] and pts[4][0] < pts[10][0] and pts[4][0] < pts[14][0]
and pts[4][0] < pts[18][0]:

            ch1 = 'A'

            if pts[4][0] > pts[6][0] and pts[4][0] < pts[10][0] and pts[4][0] < pts[14][0]
and pts[4][0] < pts[18][0] and pts[4][1] < pts[14][1] and pts[4][1] < pts[18][1] :

                ch1 = 'T'

                if pts[4][1] > pts[8][1] and pts[4][1] > pts[12][1] and pts[4][1] > pts[16][1]
and pts[4][1] > pts[20][1]:

                    ch1 = 'E'

                    if pts[4][0] > pts[6][0] and pts[4][0] > pts[10][0] and pts[4][0] > pts[14][0]
and pts[4][1] < pts[18][1]:

                        ch1 = 'M'

                        if pts[4][0] > pts[6][0] and pts[4][0] > pts[10][0] and pts[4][1] < pts[18][1]
and pts[4][1] < pts[14][1]:

                            ch1 = 'N'

                            if ch1 == 2:

                                if distance(pts[12], pts[4]) > 42:

                                    ch1 = 'C'

                                else:

                                    ch1 = 'O'

```



```

if ch1 == 3:
    if (distance(pts[8], pts[12])) > 72:
        ch1 = 'G'
    else:
        ch1 = 'H'

if ch1 == 7:
    if distance(pts[8], pts[4]) > 42:
        ch1 = 'Y'
    else:
        ch1 = 'J'

if ch1 == 4:
    ch1 = 'L'

if ch1 == 6:
    ch1 = 'X'

if ch1 == 5:
    if pts[4][0] > pts[12][0] and pts[4][0] > pts[16][0] and pts[4][0] > pts[20][0]:
        if pts[8][1] < pts[5][1]:
            ch1 = 'Z'
        else:
            ch1 = 'Q'
    else:
        ch1 = 'P'

```

```

        if ch1 == 1:
            if (pts[6][1] > pts[8][1] and pts[10][1] > pts[12][1] and pts[14][1] > pts[16][1]
and pts[18][1] > pts[20][1]):
                ch1 = 'B'

            if (pts[6][1] > pts[8][1] and pts[10][1] < pts[12][1] and pts[14][1] < pts[16][1]
and pts[18][1] < pts[20][1]):
                ch1 = 'D'

            if (pts[6][1] < pts[8][1] and pts[10][1] > pts[12][1] and pts[14][1] > pts[16][1]
and pts[18][1] > pts[20][1]):
                ch1 = 'F'

            if (pts[6][1] < pts[8][1] and pts[10][1] < pts[12][1] and pts[14][1] < pts[16][1]
and pts[18][1] > pts[20][1]):
                ch1 = 'T'

            if (pts[6][1] > pts[8][1] and pts[10][1] > pts[12][1] and pts[14][1] > pts[16][1]
and pts[18][1] < pts[20][1]):
                ch1 = 'W'

            if (pts[6][1] > pts[8][1] and pts[10][1] > pts[12][1] and pts[14][1] < pts[16][1]
and pts[18][1] < pts[20][1]) and pts[4][1] < pts[9][1]:
                ch1 = 'K'

            if ((distance(pts[8], pts[12]) - distance(pts[6], pts[10])) < 8) and (pts[6][1] >
pts[8][1] and pts[10][1] > pts[12][1] and pts[14][1] < pts[16][1] and pts[18][1] < pts[20][1]):
                ch1 = 'U'

            if ((distance(pts[8], pts[12]) - distance(pts[6], pts[10])) >= 8) and (pts[6][1] >
pts[8][1] and pts[10][1] > pts[12][1] and pts[14][1] < pts[16][1] and pts[18][1] < pts[20][1])
and (pts[4][1] > pts[9][1]):
                ch1 = 'V'

            if (pts[8][0] > pts[12][0]) and (pts[6][1] > pts[8][1] and pts[10][1] > pts[12][1]
and pts[14][1] < pts[16][1] and pts[18][1] < pts[20][1]):
                ch1 = 'R'

        if ch1 == 1 or 'E' or 'S' or 'X' or 'Y' or 'B':

```

```

        if (pts[6][1] > pts[8][1] and pts[10][1] < pts[12][1] and pts[14][1] < pts[16][1]
and pts[18][1] > pts[20][1]):

            ch1 = 'Space'

            if ch1== 'E' or 'Y' or 'B':

                if (pts[4][0] < pts[5][0] ):

                    ch1 = 'Next'

            if ch1== 'Next' or 'B' or 'C' or 'H' or 'F':

                if (pts[0][0] > pts[8][0] and pts[0][0] > pts[12][0] and pts[0][0] > pts[16][0]
and pts[0][0] > pts[20][0]) and pts[4][1]<pts[8][1] and pts[4][1]<pts[12][1] and
pts[4][1]<pts[16][1] and pts[4][1]<pts[20][1]:

                    ch1 = 'Backspace'

            print("ch1=", ch1, " ch2=", ch2, " ch3=", ch3)

            kok.append(ch1)

            ## [0->aemnst][1->bfdiuvwkr][2->co][3->gh][4->l][5->pqz][6->x][7->yj]

            if ch1 != 1:

                if (ch1,ch2) in dicttt:

                    dicttt[(ch1,ch2)] += 1

                else:

                    dicttt[(ch1,ch2)] = 1

            frame = cv2.putText(frame, "Predicted " + str(ch1), (30, 80),

                                cv2.FONT_HERSHEY_SIMPLEX,

                                3, (0, 0, 255), 2, cv2.LINE_AA)

cv2.imshow("frame", frame)

```

```

interrupt = cv2.waitKey(1)

if interrupt & 0xFF == 27:

    # esc key

    break

except Exception:

    print("==", traceback.format_exc())

dicttt = {key: val for key, val in sorted(dicttt.items(), key = lambda ele: ele[1], reverse =
True)}

print(dicttt)

print(set(kok))

capture.release()

cv2.destroyAllWindows()

```

final_pred.py

```

# Importing Libraries

import numpy as np

import math

import cv2

import os, sys

import traceback

import pyttsx3

from keras.models import load_model

from cvzone.HandTrackingModule import HandDetector

from string import ascii_uppercase

import enchant

ddd=enchant.Dict("en-US")

hd = HandDetector(maxHands=1)

hd2 = HandDetector(maxHands=1)

```

```

import tkinter as tk

from PIL import Image, ImageTk

offset=29

os.environ["THEANO_FLAGS"] = "device=cuda, assert_no_cpu_op=True"

# Application :

class Application:

    def __init__(self):

        self.vs = cv2.VideoCapture(0)

        self.current_image = None

        self.model = load_model('cnn8grps_rad1_model.h5')

        self.speak_engine=pyttsx3.init()

        self.speak_engine.setProperty("rate",100)

        voices=self.speak_engine.getProperty("voices")

        self.speak_engine.setProperty("voice",voices[0].id)


        self.ct = {}

        self.ct['blank'] = 0

        self.blank_flag = 0

        self.space_flag=False

        self.next_flag=True

        self.prev_char=""

        self.count=-1

        self.ten_prev_char=[]

        for i in range(10):

            self.ten_prev_char.append(" ")

```

```

for i in ascii_uppercase:

self.ct[i] = 0

print("Loaded model from disk")

self.root = tk.Tk()

self.root.title("Sign Language To Text Conversion")

self.root.protocol('WM_DELETE_WINDOW', self.destructor)

self.root.geometry("1300x700")

self.panel = tk.Label(self.root)

self.panel.place(x=100, y=3, width=480, height=640)


self.panel2 = tk.Label(self.root) # initialize image panel
self.panel2.place(x=700, y=115, width=400, height=400)


self.T = tk.Label(self.root)

self.T.place(x=60, y=5)

self.T.config(text="Sign Language To Text Conversion", font=("Courier", 30, "bold"))


self.panel3 = tk.Label(self.root) # Current Symbol
self.panel3.place(x=280, y=585)


self.T1 = tk.Label(self.root)

self.T1.place(x=10, y=580)

self.T1.config(text="Character :", font=("Courier", 30, "bold"))


self.panel5 = tk.Label(self.root) # Sentence
self.panel5.place(x=260, y=632)

```

```

self.T3 = tk.Label(self.root)
self.T3.place(x=10, y=632)
self.T3.config(text="Sentence :", font=("Courier", 30, "bold"))

self.T4 = tk.Label(self.root)
self.T4.place(x=10, y=700)
self.T4.config(text="Suggestions :", fg="red", font=("Courier", 30, "bold"))

self.b1=tk.Button(self.root)
self.b1.place(x=390,y=700)

self.b2 = tk.Button(self.root)
self.b2.place(x=590, y=700)

self.b3 = tk.Button(self.root)
self.b3.place(x=790, y=700)

self.b4 = tk.Button(self.root)
self.b4.place(x=990, y=700)

self.speak = tk.Button(self.root)
self.speak.place(x=1305, y=630)
self.speak.config(text="Speak", font=("Courier", 20), wraplength=100,
command=self.speak_fun)

self.clear = tk.Button(self.root)
self.clear.place(x=1205, y=630)

```

```

        self.clear.config(text="Clear", font=("Courier", 20), wraplength=100,
command=self.clear_fun)

self.str = " "

self.ccc=0

self.word = " "

self.current_symbol = "C"

self.photo = "Empty"


self.word1=" "

self.word2 = " "

self.word3 = " "

self.word4 = " "


self.video_loop()


def video_loop(self):
try:
ok, frame = self.vs.read()

cv2image = cv2.flip(frame, 1)

if cv2image.any:

    hands = hd.findHands(cv2image, draw=False, flipType=True)

    cv2image_copy=np.array(cv2image)

    cv2image = cv2.cvtColor(cv2image, cv2.COLOR_BGR2RGB)

    self.current_image = Image.fromarray(cv2image)

    imgtk = ImageTk.PhotoImage(image=self.current_image)

    self.panel.imgtk = imgtk

    self.panel.config(image=imgtk)

```



```

if hands[0]:
    hand = hands[0]
    map = hand[0]
    x, y, w, h=map['bbox']
    image = cv2image_copy[y - offset:y + h + offset, x - offset:x + w + offset]

white = cv2.imread("white.jpg")
# img_final=img_final1=img_final2=0
if image.all:
    handz = hd2.findHands(image, draw=False, flipType=True)
    self.ccc += 1
    if handz[0]:
        hand = handz[0]
        handmap=hand[0]
        self.pts = handmap['lmList']
        # x1,y1,w1,h1=hand['bbox']

        os = ((400 - w) // 2) - 15
        os1 = ((400 - h) // 2) - 15
        for t in range(0, 4, 1):
            cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1), (self.pts[t +
1][0] + os, self.pts[t + 1][1] + os1),
                (0, 255, 0), 3)
        for t in range(5, 8, 1):
            cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1), (self.pts[t +
1][0] + os, self.pts[t + 1][1] + os1),
                (0, 255, 0), 3)

```

```

        for t in range(9, 12, 1):
            cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1), (self.pts[t +
1][0] + os, self.pts[t + 1][1] + os1),
                    (0, 255, 0), 3)

        for t in range(13, 16, 1):
            cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1), (self.pts[t +
1][0] + os, self.pts[t + 1][1] + os1),
                    (0, 255, 0), 3)

        for t in range(17, 20, 1):
            cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1), (self.pts[t +
1][0] + os, self.pts[t + 1][1] + os1),
                    (0, 255, 0), 3)

            cv2.line(white, (self.pts[5][0] + os, self.pts[5][1] + os1), (self.pts[9][0]
+ os, self.pts[9][1] + os1), (0, 255, 0),
                    3)

            cv2.line(white, (self.pts[9][0] + os, self.pts[9][1] + os1),
(self.pts[13][0] + os, self.pts[13][1] + os1), (0, 255, 0),
                    3)

            cv2.line(white, (self.pts[13][0] + os, self.pts[13][1] + os1),
(self.pts[17][0] + os, self.pts[17][1] + os1),
                    (0, 255, 0), 3)

            cv2.line(white, (self.pts[0][0] + os, self.pts[0][1] + os1), (self.pts[5][0]
+ os, self.pts[5][1] + os1), (0, 255, 0),
                    3)

            cv2.line(white, (self.pts[0][0] + os, self.pts[0][1] + os1),
(self.pts[17][0] + os, self.pts[17][1] + os1), (0, 255, 0),
                    3)

        for i in range(21):
            cv2.circle(white, (self.pts[i][0] + os, self.pts[i][1] + os1), 2, (0, 0, 255), 1)

```

```

        res=white
        self.predict(res)
        self.current_image2 = Image.fromarray(res)
        imgtk = ImageTk.PhotoImage(image=self.current_image2)
        self.panel2.imgtk = imgtk
        self.panel2.config(image=imgtk)
        self.panel3.config(text=self.current_symbol, font=("Courier", 30))
        #self.panel4.config(text=self.word, font=("Courier", 30))
        self.b1.config(text=self.word1, font=("Courier", 20), wraplength=825,
command=self.action1)
        self.b2.config(text=self.word2, font=("Courier", 20), wraplength=825,
command=self.action2)
        self.b3.config(text=self.word3, font=("Courier", 20), wraplength=825,
command=self.action3)
        self.b4.config(text=self.word4, font=("Courier", 20), wraplength=825,
command=self.action4)

        self.panel5.config(text=self.str, font=("Courier", 30), wraplength=1025)
    except Exception:
        print(Exception.__traceback__)
        hands = hd.findHands(cv2image, draw=False, flipType=True)
        cv2image_copy=np.array(cv2image)
        cv2image = cv2.cvtColor(cv2image, cv2.COLOR_BGR2RGB)
        self.current_image = Image.fromarray(cv2image)
        imgtk = ImageTk.PhotoImage(image=self.current_image)
        self.panel.imgtk = imgtk
        self.panel.config(image=imgtk)

```

```

if hands:

    # #print("----- lmList=",hands[1])

    hand = hands[0]

    x, y, w, h = hand['bbox']

    image = cv2image_copy[y - offset:y + h + offset, x - offset:x + w + offset]

    white = cv2.imread("C:\\Users\\PycharmProjects\\pythonProject\\white.jpg")

    # img_final=img_final1=img_final2=0

    handz = hd2.findHands(image, draw=False, flipType=True)

    print(" ", self.ccc)

    self.ccc += 1

    if handz:

        hand = handz[0]

        self.pts = hand['lmList']

        # x1,y1,w1,h1=hand['bbox']


    os = ((400 - w) // 2) - 15

    os1 = ((400 - h) // 2) - 15

    for t in range(0, 4, 1):

        cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1), (self.pts[t + 1][0] + os,
self.pts[t + 1][1] + os1),

                (0, 255, 0), 3)

    for t in range(5, 8, 1):

        cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1), (self.pts[t + 1][0] + os,
self.pts[t + 1][1] + os1),

                (0, 255, 0), 3)

    for t in range(9, 12, 1):

```

```

        cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1), (self.pts[t + 1][0] + os,
self.pts[t + 1][1] + os1),

                (0, 255, 0), 3)

        for t in range(13, 16, 1):

            cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1), (self.pts[t + 1][0] + os,
self.pts[t + 1][1] + os1),

                    (0, 255, 0), 3)

            for t in range(17, 20, 1):

                cv2.line(white, (self.pts[t][0] + os, self.pts[t][1] + os1), (self.pts[t + 1][0] + os,
self.pts[t + 1][1] + os1),

                        (0, 255, 0), 3)

                cv2.line(white, (self.pts[5][0] + os, self.pts[5][1] + os1), (self.pts[9][0] + os,
self.pts[9][1] + os1), (0, 255, 0),

                        3)

                cv2.line(white, (self.pts[9][0] + os, self.pts[9][1] + os1), (self.pts[13][0] + os,
self.pts[13][1] + os1), (0, 255, 0),

                        3)

                cv2.line(white, (self.pts[13][0] + os, self.pts[13][1] + os1), (self.pts[17][0] +
os, self.pts[17][1] + os1),

                        (0, 255, 0), 3)

                cv2.line(white, (self.pts[0][0] + os, self.pts[0][1] + os1), (self.pts[5][0] + os,
self.pts[5][1] + os1), (0, 255, 0),

                        3)

                cv2.line(white, (self.pts[0][0] + os, self.pts[0][1] + os1), (self.pts[17][0] + os,
self.pts[17][1] + os1), (0, 255, 0),

                        3)

        for i in range(21):

            cv2.circle(white, (self.pts[i][0] + os, self.pts[i][1] + os1), 2, (0, 0, 255), 1)

        res=white

```

```

        self.predict(res)

        self.current_image2 = Image.fromarray(res)

        imgtk = ImageTk.PhotoImage(image=self.current_image2)

        self.panel2.imgtk = imgtk

        self.panel2.config(image=imgtk)

        self.panel3.config(text=self.current_symbol, font=("Courier", 30))

        #self.panel4.config(text=self.word, font=("Courier", 30))

        self.b1.config(text=self.word1, font=("Courier", 20), wraplength=825,
command=self.action1)

        self.b2.config(text=self.word2, font=("Courier", 20), wraplength=825,
command=self.action2)

        self.b3.config(text=self.word3, font=("Courier", 20), wraplength=825,
command=self.action3)

        self.b4.config(text=self.word4, font=("Courier", 20), wraplength=825,
command=self.action4)

self.panel5.config(text=self.str, font=("Courier", 30), wraplength=1025)

except Exception:

    print("==", traceback.format_exc())

finally:

    self.root.after(1, self.video_loop)

def distance(self,x,y):

    return math.sqrt(((x[0] - y[0]) ** 2) + ((x[1] - y[1]) ** 2))

def action1(self):

    idx_space = self.str.rfind(" ")

    idx_word = self.str.find(self.word, idx_space)

```

```

last_idx = len(self.str)
self.str = self.str[:idx_word]
self.str = self.str + self.word1.upper()

def action2(self):
    idx_space = self.str.rfind(" ")
    idx_word = self.str.find(self.word, idx_space)
    last_idx = len(self.str)
    self.str=self.str[:idx_word]
    self.str=self.str+self.word2.upper()
    #self.str[idx_word:last_idx] = self.word2

def action3(self):
    idx_space = self.str.rfind(" ")
    idx_word = self.str.find(self.word, idx_space)
    last_idx = len(self.str)
    self.str = self.str[:idx_word]
    self.str = self.str + self.word3.upper()

def action4(self):
    idx_space = self.str.rfind(" ")
    idx_word = self.str.find(self.word, idx_space)
    last_idx = len(self.str)
    self.str = self.str[:idx_word]
    self.str = self.str + self.word4.upper()

def speak_fun(self):
    self.speak_engine.say(self.str)
    self.speak_engine.runAndWait()

```

```

def clear_fun(self):
    self.str=""
    self.word1 = ""
    self.word2 = ""
    self.word3 = ""
    self.word4 = ""

def predict(self, test_image):
    white=test_image
    white = white.reshape(1, 400, 400, 3)
    prob = np.array(self.model.predict(white)[0], dtype='float32')
    ch1 = np.argmax(prob, axis=0)
    prob[ch1] = 0
    ch2 = np.argmax(prob, axis=0)
    prob[ch2] = 0
    ch3 = np.argmax(prob, axis=0)
    prob[ch3] = 0

    pl = [ch1, ch2]

    # condition for [Aemnst]
    l = [[5, 2], [5, 3], [3, 5], [3, 6], [3, 0], [3, 2], [6, 4], [6, 1], [6, 2], [6, 6], [6, 7], [6, 0],
[6, 5],
    [4, 1], [1, 0], [1, 1], [6, 3], [1, 6], [5, 6], [5, 1], [4, 5], [1, 4], [1, 5], [2, 0], [2, 6], [4, 6],
    [1, 0], [5, 7], [1, 6], [6, 1], [7, 6], [2, 5], [7, 1], [5, 4], [7, 0], [7, 5], [7, 2]]
    if pl in l:
        if (self.pts[6][1] < self.pts[8][1] and self.pts[10][1] < self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and self.pts[18][1] < self.pts[20][

```



```

1)):
    ch1 = 0

# condition for [o][s]
l = [[2, 2], [2, 1]]
if pl in l:
    if (self.pts[5][0] < self.pts[4][0]):
        ch1 = 0
        print("+++++")
        # print("00000")

# condition for [c0][aemnst]
l = [[0, 0], [0, 6], [0, 2], [0, 5], [0, 1], [0, 7], [5, 2], [7, 6], [7, 1]]
pl = [ch1, ch2]
if pl in l:
    if (self.pts[0][0] > self.pts[8][0] and self.pts[0][0] > self.pts[4][0] and self.pts[0][0] >
self.pts[12][0] and self.pts[0][0] > self.pts[16][
0] and self.pts[0][0] > self.pts[20][0]) and self.pts[5][0] > self.pts[4][0]:
        ch1 = 2

# condition for [c0][aemnst]
l = [[6, 0], [6, 6], [6, 2]]
pl = [ch1, ch2]
if pl in l:
    if self.distance(self.pts[8], self.pts[16]) < 52:
        ch1 = 2

```

```

# condition for [gh][bdfikruvw]

l = [[1, 4], [1, 5], [1, 6], [1, 3], [1, 0]]

pl = [ch1, ch2]

if pl in l:

    if self.pts[6][1] > self.pts[8][1] and self.pts[14][1] < self.pts[16][1] and self.pts[18][1]
    < self.pts[20][1] and self.pts[0][0] < self.pts[8][
        0] and self.pts[0][0] < self.pts[12][0] and self.pts[0][0] < self.pts[16][0] and
        self.pts[0][0] < self.pts[20][0]:

        ch1 = 3

# con for [gh][l]

l = [[4, 6], [4, 1], [4, 5], [4, 3], [4, 7]]

pl = [ch1, ch2]

if pl in l:

    if self.pts[4][0] > self.pts[0][0]:

        ch1 = 3

# con for [gh][pqz]

l = [[5, 3], [5, 0], [5, 7], [5, 4], [5, 2], [5, 1], [5, 5]]

pl = [ch1, ch2]

if pl in l:

    if self.pts[2][1] + 15 < self.pts[16][1]:

        ch1 = 3

# con for [l][x]

l = [[6, 4], [6, 1], [6, 2]]

pl = [ch1, ch2]

```

```

if pl in l:
    if self.distance(self.pts[4], self.pts[11]) > 55:
        ch1 = 4

# con for [l][d]
l = [[1, 4], [1, 6], [1, 1]]
pl = [ch1, ch2]
if pl in l:
    if (self.distance(self.pts[4], self.pts[11]) > 50) and (
        self.pts[6][1] > self.pts[8][1] and self.pts[10][1] < self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and self.pts[18][1] <
        self.pts[20][1]):
        ch1 = 4

# con for [l][gh]
l = [[3, 6], [3, 4]]
pl = [ch1, ch2]
if pl in l:
    if (self.pts[4][0] < self.pts[0][0]):
        ch1 = 4

# con for [l][c0]
l = [[2, 2], [2, 5], [2, 4]]
pl = [ch1, ch2]
if pl in l:
    if (self.pts[1][0] < self.pts[12][0]):
        ch1 = 4

```

```

# con for [l][c0]
l = [[2, 2], [2, 5], [2, 4]]
pl = [ch1, ch2]
if pl in l:
if (self.pts[1][0] < self.pts[12][0]):
    ch1 = 4

# con for [gh][z]
l = [[3, 6], [3, 5], [3, 4]]
pl = [ch1, ch2]
if pl in l:
    if (self.pts[6][1] > self.pts[8][1] and self.pts[10][1] < self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and self.pts[18][1] < self.pts[20][
    1]) and self.pts[4][1] > self.pts[10][1]:
        ch1 = 5

# con for [gh][pq]
l = [[3, 2], [3, 1], [3, 6]]
pl = [ch1, ch2]
if pl in l:
    if self.pts[4][1] + 17 > self.pts[8][1] and self.pts[4][1] + 17 > self.pts[12][1] and
self.pts[4][1] + 17 > self.pts[16][1] and self.pts[4][
    1] + 17 > self.pts[20][1]:
        ch1 = 5

# con for [l][pqz]
l = [[4, 4], [4, 5], [4, 2], [7, 5], [7, 6], [7, 0]]

```

```

pl = [ch1, ch2]

if pl in l:

    if self.pts[4][0] > self.pts[0][0]:

        ch1 = 5


# con for [pqz][aemnst]

l = [[0, 2], [0, 6], [0, 1], [0, 5], [0, 0], [0, 7], [0, 4], [0, 3], [2, 7]]

pl = [ch1, ch2]

if pl in l:

    if self.pts[0][0] < self.pts[8][0] and self.pts[0][0] < self.pts[12][0] and self.pts[0][0] <
self.pts[16][0] and self.pts[0][0] < self.pts[20][0]:

        ch1 = 5


# con for [pqz][yj]

l = [[5, 7], [5, 2], [5, 6]]

pl = [ch1, ch2]

if pl in l:

    if self.pts[3][0] < self.pts[0][0]:

        ch1 = 7


# con for [l][yj]

l = [[4, 6], [4, 2], [4, 4], [4, 1], [4, 5], [4, 7]]

pl = [ch1, ch2]

if pl in l:

    if self.pts[6][1] < self.pts[8][1]:

        ch1 = 7

```

```

# con for [x][yj]
l = [[6, 7], [0, 7], [0, 1], [0, 0], [6, 4], [6, 6], [6, 5], [6, 1]]
pl = [ch1, ch2]
if pl in l:
if self.pts[18][1] > self.pts[20][1]:
    ch1 = 7

# condition for [x][aemnst]
l = [[0, 4], [0, 2], [0, 3], [0, 1], [0, 6]]
pl = [ch1, ch2]
if pl in l:
if self.pts[5][0] > self.pts[16][0]:
    ch1 = 6

# condition for [yj][x]
print("2222 ch1=+++++", ch1, ",", ch2)
l = [[7, 2]]
pl = [ch1, ch2]
if pl in l:
if self.pts[18][1] < self.pts[20][1] and self.pts[8][1] < self.pts[10][1]:
    ch1 = 6

# condition for [c0][x]
l = [[2, 1], [2, 2], [2, 6], [2, 7], [2, 0]]
pl = [ch1, ch2]
if pl in l:
if self.distance(self.pts[8], self.pts[16]) > 50:

```

```

        ch1 = 6

# con for [l][x]
l = [[4, 6], [4, 2], [4, 1], [4, 4]]
pl = [ch1, ch2]
if pl in l:
if self.distance(self.pts[4], self.pts[11]) < 60:
        ch1 = 6

# con for [x][d]
l = [[1, 4], [1, 6], [1, 0], [1, 2]]
pl = [ch1, ch2]
if pl in l:
if self.pts[5][0] - self.pts[4][0] - 15 > 0:
        ch1 = 6

# con for [b][pqz]
l = [[5, 0], [5, 1], [5, 4], [5, 5], [5, 6], [6, 1], [7, 6], [0, 2], [7, 1], [7, 4], [6, 6], [7, 2],
[5, 0],
[6, 3], [6, 4], [7, 5], [7, 2]]
pl = [ch1, ch2]
if pl in l:
        if (self.pts[6][1] > self.pts[8][1] and self.pts[10][1] > self.pts[12][1] and
self.pts[14][1] > self.pts[16][1] and self.pts[18][1] > self.pts[20][
1]):
                ch1 = 1

# con for [f][pqz]

```

```

l = [[6, 1], [6, 0], [0, 3], [6, 4], [2, 2], [0, 6], [6, 2], [7, 6], [4, 6], [4, 1], [4, 2], [0, 2],
[7, 1],
[7, 4], [6, 6], [7, 2], [7, 5], [7, 2]]

pl = [ch1, ch2]

if pl in l:

    if (self.pts[6][1] < self.pts[8][1] and self.pts[10][1] > self.pts[12][1] and
self.pts[14][1] > self.pts[16][1] and

        self.pts[18][1] > self.pts[20][1]):

            ch1 = 1

l = [[6, 1], [6, 0], [4, 2], [4, 1], [4, 6], [4, 4]]

pl = [ch1, ch2]

if pl in l:

    if (self.pts[10][1] > self.pts[12][1] and self.pts[14][1] > self.pts[16][1] and

        self.pts[18][1] > self.pts[20][1]):

            ch1 = 1

# con for [d][pqz]

fg = 19

# print("_____ch1=",ch1," ch2=",ch2)

l = [[5, 0], [3, 4], [3, 0], [3, 1], [3, 5], [5, 5], [5, 4], [5, 1], [7, 6]]

pl = [ch1, ch2]

if pl in l:

    if ((self.pts[6][1] > self.pts[8][1] and self.pts[10][1] < self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and

        self.pts[18][1] < self.pts[20][1]) and (self.pts[2][0] < self.pts[0][0]) and
self.pts[4][1] > self.pts[14][1]):

            ch1 = 1

```



```

l = [[4, 1], [4, 2], [4, 4]]

pl = [ch1, ch2]

if pl in l:

    if (self.distance(self.pts[4], self.pts[11]) < 50) and (

        self.pts[6][1] > self.pts[8][1] and self.pts[10][1] < self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and self.pts[18][1] <

        self.pts[20][1]):

        ch1 = 1

l = [[3, 4], [3, 0], [3, 1], [3, 5], [3, 6]]

pl = [ch1, ch2]

if pl in l:

    if ((self.pts[6][1] > self.pts[8][1] and self.pts[10][1] < self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and

        self.pts[18][1] < self.pts[20][1]) and (self.pts[2][0] < self.pts[0][0]) and
self.pts[14][1] < self.pts[4][1]):

        ch1 = 1

l = [[6, 6], [6, 4], [6, 1], [6, 2]]

pl = [ch1, ch2]

if pl in l:

    if self.pts[5][0] - self.pts[4][0] - 15 < 0:

        ch1 = 1

# con for [i][pqz]

l = [[5, 4], [5, 5], [5, 1], [0, 3], [0, 7], [5, 0], [0, 2], [6, 2], [7, 5], [7, 1], [7, 6], [7, 7]]

pl = [ch1, ch2]

if pl in l:

```

```

        if ((self.pts[6][1] < self.pts[8][1] and self.pts[10][1] < self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and

            self.pts[18][1] > self.pts[20][1])):

            ch1 = 1

            # con for [yj][bfdi]

            l = [[1, 5], [1, 7], [1, 1], [1, 6], [1, 3], [1, 0]]

            pl = [ch1, ch2]

            if pl in l:

                if (self.pts[4][0] < self.pts[5][0] + 15) and (

                    (self.pts[6][1] < self.pts[8][1] and self.pts[10][1] < self.pts[12][1] and self.pts[14][1]
< self.pts[16][1] and

                        self.pts[18][1] > self.pts[20][1])):

                            ch1 = 7

                                # con for [uvr]

                                l = [[5, 5], [5, 0], [5, 4], [5, 1], [4, 6], [4, 1], [7, 6], [3, 0], [3, 5]]

                                pl = [ch1, ch2]

                                if pl in l:

                                    if ((self.pts[6][1] > self.pts[8][1] and self.pts[10][1] > self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and

                                        self.pts[18][1] < self.pts[20][1])) and self.pts[4][1] > self.pts[14][1]:

                                            ch1 = 1

                                                # con for [w]

                                                fg = 13

                                                l = [[3, 5], [3, 0], [3, 6], [5, 1], [4, 1], [2, 0], [5, 0], [5, 5]]

                                                pl = [ch1, ch2]

                                                if pl in l:

```

```

        if not (self.pts[0][0] + fg < self.pts[8][0] and self.pts[0][0] + fg < self.pts[12][0] and
self.pts[0][0] + fg < self.pts[16][0] and

                self.pts[0][0] + fg < self.pts[20][0]) and not (

                self.pts[0][0] > self.pts[8][0] and self.pts[0][0] > self.pts[12][0] and
self.pts[0][0] > self.pts[16][0] and self.pts[0][0] > self.pts[20][

                0]) and self.distance(self.pts[4], self.pts[11]) < 50:

            ch1 = 1

# con for [w]

l = [[5, 0], [5, 5], [0, 1]]

pl = [ch1, ch2]

if pl in l:

    if self.pts[6][1] > self.pts[8][1] and self.pts[10][1] > self.pts[12][1] and self.pts[14][1]
> self.pts[16][1]:

        ch1 = 1

# -----condn for 8 groups ends

# -----condn for subgroups starts

#

if ch1 == 0:

    ch1 = 'S'

    if self.pts[4][0] < self.pts[6][0] and self.pts[4][0] < self.pts[10][0] and self.pts[4][0] <
self.pts[14][0] and self.pts[4][0] < self.pts[18][0]:

        ch1 = 'A'

    if self.pts[4][0] > self.pts[6][0] and self.pts[4][0] < self.pts[10][0] and self.pts[4][0] <
self.pts[14][0] and self.pts[4][0] < self.pts[18][

        0] and self.pts[4][1] < self.pts[14][1] and self.pts[4][1] < self.pts[18][1]:

            ch1 = 'T'

    if self.pts[4][1] > self.pts[8][1] and self.pts[4][1] > self.pts[12][1] and self.pts[4][1] >
self.pts[16][1] and self.pts[4][1] > self.pts[20][1]:

        ch1 = 'E'

```

```
        if self.pts[4][0] > self.pts[6][0] and self.pts[4][0] > self.pts[10][0] and self.pts[4][0] >
self.pts[14][0] and self.pts[4][1] < self.pts[18][1]:
```

```
            ch1 = 'M'
```

```
        if self.pts[4][0] > self.pts[6][0] and self.pts[4][0] > self.pts[10][0] and self.pts[4][1] <
self.pts[18][1] and self.pts[4][1] < self.pts[14][1]:
```

```
            ch1 = 'N'
```

```
    if ch1 == 2:
```

```
        if self.distance(self.pts[12], self.pts[4]) > 42:
```

```
            ch1 = 'C'
```

```
        else:
```

```
            ch1 = 'O'
```

```
    if ch1 == 3:
```

```
        if (self.distance(self.pts[8], self.pts[12])) > 72:
```

```
            ch1 = 'G'
```

```
        else:
```

```
            ch1 = 'H'
```

```
    if ch1 == 7:
```

```
        if self.distance(self.pts[8], self.pts[4]) > 42:
```

```
            ch1 = 'Y'
```

```
        else:
```

```
            ch1 = 'J'
```

```
    if ch1 == 4:
```

```
        ch1 = 'L'
```

```
    if ch1 == 6:
```

```
        ch1 = 'X'
```

```

    if ch1 == 5:

        if self.pts[4][0] > self.pts[12][0] and self.pts[4][0] > self.pts[16][0] and self.pts[4][0]
> self.pts[20][0]:

            if self.pts[8][1] < self.pts[5][1]:

                ch1 = 'Z'

            else:

                ch1 = 'Q'

        else:

            ch1 = 'P'

    if ch1 == 1:

        if (self.pts[6][1] > self.pts[8][1] and self.pts[10][1] > self.pts[12][1] and
self.pts[14][1] > self.pts[16][1] and self.pts[18][1] > self.pts[20][

            1)):

                ch1 = 'B'

        if (self.pts[6][1] > self.pts[8][1] and self.pts[10][1] < self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and self.pts[18][1] < self.pts[20][

            1)):

                ch1 = 'D'

        if (self.pts[6][1] < self.pts[8][1] and self.pts[10][1] > self.pts[12][1] and
self.pts[14][1] > self.pts[16][1] and self.pts[18][1] > self.pts[20][

            1)):

                ch1 = 'F'

        if (self.pts[6][1] < self.pts[8][1] and self.pts[10][1] < self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and self.pts[18][1] > self.pts[20][

            1)):

                ch1 = 'T'

        if (self.pts[6][1] > self.pts[8][1] and self.pts[10][1] > self.pts[12][1] and
self.pts[14][1] > self.pts[16][1] and self.pts[18][1] < self.pts[20][

            1)):

                ch1 = 'W'

```

```

        if (self.pts[6][1] > self.pts[8][1] and self.pts[10][1] > self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and self.pts[18][1] < self.pts[20][

            1]) and self.pts[4][1] < self.pts[9][1]:

                ch1 = 'K'

            if ((self.distance(self.pts[8], self.pts[12]) - self.distance(self.pts[6], self.pts[10])) < 8)
and (

                self.pts[6][1] > self.pts[8][1] and self.pts[10][1] > self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and self.pts[18][1] <

                    self.pts[20][1]):

                        ch1 = 'U'

            if ((self.distance(self.pts[8], self.pts[12]) - self.distance(self.pts[6], self.pts[10])) >= 8)
and (

                self.pts[6][1] > self.pts[8][1] and self.pts[10][1] > self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and self.pts[18][1] <

                    self.pts[20][1]) and (self.pts[4][1] > self.pts[9][1]):

                        ch1 = 'V'

            if (self.pts[8][0] > self.pts[12][0]) and (

                self.pts[6][1] > self.pts[8][1] and self.pts[10][1] > self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and self.pts[18][1] <

                    self.pts[20][1]):

                        ch1 = 'R'

            if ch1 == 1 or ch1 == 'E' or ch1 == 'S' or ch1 == 'X' or ch1 == 'Y' or ch1 == 'B':

                if (self.pts[6][1] > self.pts[8][1] and self.pts[10][1] < self.pts[12][1] and
self.pts[14][1] < self.pts[16][1] and self.pts[18][1] > self.pts[20][1]):

                    ch1=""

                print(self.pts[4][0] < self.pts[5][0])

                if ch1 == 'E' or ch1=='Y' or ch1=='B':

                    if (self.pts[4][0] < self.pts[5][0]) and (self.pts[6][1] > self.pts[8][1] and self.pts[10][1]
> self.pts[12][1] and self.pts[14][1] > self.pts[16][1] and self.pts[18][1] > self.pts[20][1]):

                        ch1="next"

```

```
if ch1 == 'Next' or 'B' or 'C' or 'H' or 'F' or 'X':
```

```
    if (self.pts[0][0] > self.pts[8][0] and self.pts[0][0] > self.pts[12][0] and self.pts[0][0] >
        self.pts[16][0] and self.pts[0][0] > self.pts[20][0]) and (self.pts[4][1] < self.pts[8][1] and
        self.pts[4][1] < self.pts[12][1] and self.pts[4][1] < self.pts[16][1] and self.pts[4][1] <
        self.pts[20][1]) and (self.pts[4][1] < self.pts[6][1] and self.pts[4][1] < self.pts[10][1] and
        self.pts[4][1] < self.pts[14][1] and self.pts[4][1] < self.pts[18][1]):
```

```
        ch1 = 'Backspace'
```

```
if ch1=="next" and self.prev_char!="next":
```

```
if self.ten_prev_char[(self.count-2)%10]!="next":
```

```
    if self.ten_prev_char[(self.count-2)%10]=="Backspace":
```

```
        self.str=self.str[0:-1]
```

```
    else:
```

```
        if self.ten_prev_char[(self.count - 2) % 10] != "Backspace":
```

```
            self.str = self.str + self.ten_prev_char[(self.count-2)%10]
```

```
else:
```

```
    if self.ten_prev_char[(self.count - 0) % 10] != "Backspace":
```

```
        self.str = self.str + self.ten_prev_char[(self.count - 0) % 10]
```

```
if ch1==" " and self.prev_char!=" ":
```

```
self.str = self.str + " "
```

```
self.prev_char=ch1
```

```
self.current_symbol=ch1
```

```
self.count += 1
```

```
self.ten_prev_char[self.count%10]=ch1
```

```
if len(self.str.strip())!=0:
```

```
st=self.str.rfind(" ")
```

```

ed=len(self.str)
word=self.str[st+1:ed]
self.word=word
if len(word.strip())!=0:
    ddd.check(word)
    lenn = len(ddd.suggest(word))
    if lenn >= 4:
        self.word4 = ddd.suggest(word)[3]
    if lenn >= 3:
        self.word3 = ddd.suggest(word)[2]
    if lenn >= 2:
        self.word2 = ddd.suggest(word)[1]
    if lenn >= 1:
        self.word1 = ddd.suggest(word)[0]
else:
    self.word1 = " "
    self.word2 = " "
    self.word3 = " "
    self.word4 = " "

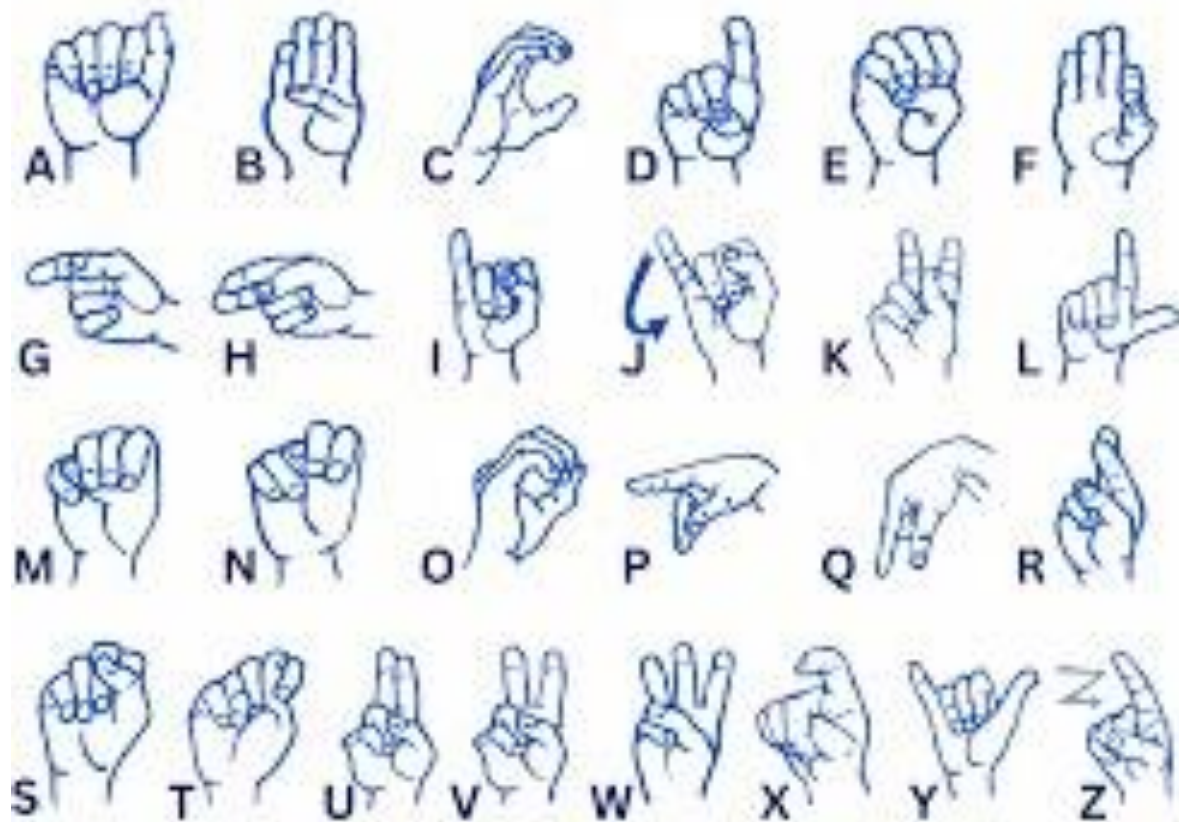
def destructor(self):
    print(self.ten_prev_char)
    self.root.destroy()
    self.vs.release()
    cv2.destroyAllWindows()

print("Starting Application...")
(Application()).root.mainloop()

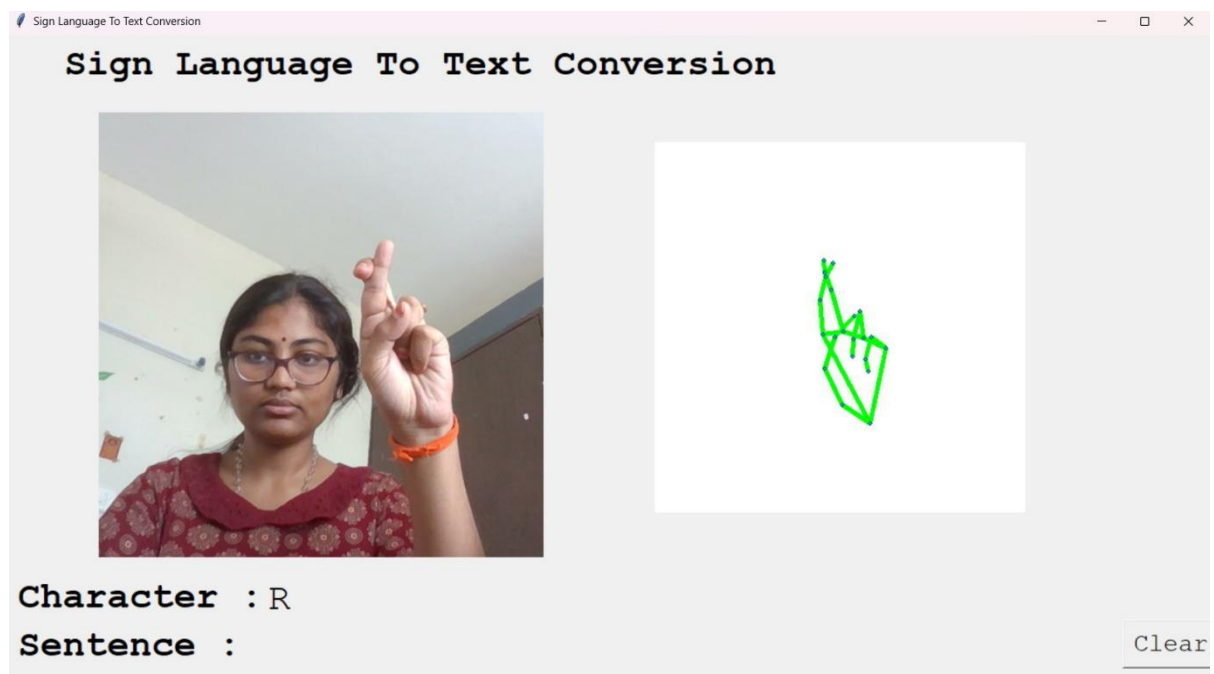
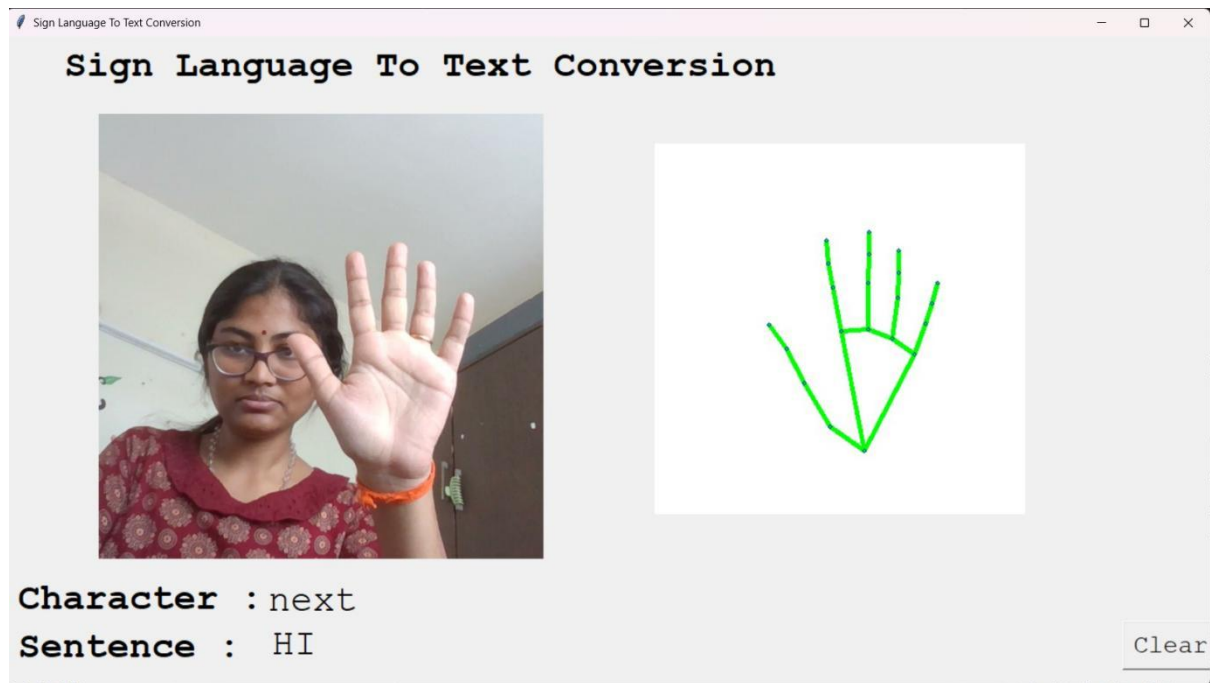
```


Screenshots:

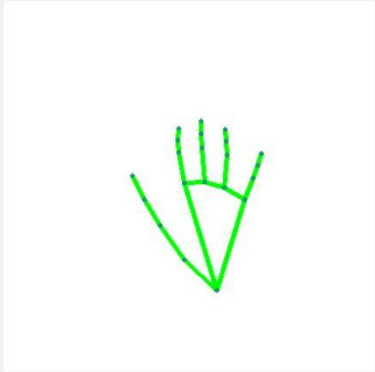
American Sign Language



Predictions



Sign Language To Text Conversion



Character : B
Sentence : OA

Clear

Sign Language To Text Conversion



Character : Backspace
Sentence : OA

Clear